15.095: Machine Learning Under a Modern Optimization Lens **Fall 2026**

Homework 2: Due October 5

Hand in: **pdf** upload to Gradescope. Please append any Julia code **at the end** of the whole pdf.

Note: this homework covers lectures 4–7. We recommend attempting questions after the relevant content has been covered in class. Total: 100 points.

Question 1: True/False (30 points)

Please classify the following statements as true or false and justify your answer. We will assign 1 point for correctly classifying the answer, and 5 points for the validity of the justification/counterexample.

(a) (6 points) Consider the non-regularized convex regression formulation

$$\min_{\boldsymbol{\theta}\in\mathbb{R}^n,\ \{\boldsymbol{\xi}_i\in\mathbb{R}^p\}} \ \frac{1}{2}\sum_{i=1}^{n}(y_i-\theta_i)^2 \quad \text{s.t.} \quad \theta_i+\boldsymbol{\xi}_i^\top(\mathbf{x}_j-\mathbf{x}_i)\le\theta_j \ \ \forall i,j\in[n].$$

Assume we have data points $(\mathbf{x}_i, y_i = f(\mathbf{x}_i))$ for $i \in [n]$, $n > 1$, but we are told the underlying f is **non-convex**. Then applying this formulation to approximate f always gives a non-zero objective.

(b) (6 points) We are solving a variation of a regression problem with residuals $r_i = |y_i - \mathbf{x}_i^\top\boldsymbol{\beta}|$, $i \in [n]$. We want to find $\boldsymbol{\beta}$ such that the **sum of the k lowest residuals** is minimized. This problem can be formulated as a linear MIO.

(c) (6 points) Consider the holistic regression MIO, which uses binary indicators $z_j \in \{0, 1\}$, $j \in [p]$, together with a big-M constraint $-Mz_j \le \beta_j \le Mz_j$ to select features. Suppose two variables $\mathbf{x}_i$ and $\mathbf{x}_j$ are highly correlated. We could equivalently enforce this the way we enforce group sparsity, by adding i, j to the same group GS_m and imposing $z_i = z_j$, instead of the pairwise-collinearity constraint $z_i + z_j \le 1$.

(d) (6 points) Consider logistic regression with row-wise uncertainty in the features, $\mathcal{U}_x = \{\boldsymbol{\Delta}X : \|\boldsymbol{\Delta}\mathbf{x}_i\|_q \le \rho,\ i = 1, \ldots, n\}$. The robust counterpart places the robustness term inside the exponent, using the same norm that defines the uncertainty set:

$$\max_{\boldsymbol{\beta},\beta_0} \ -\sum_{i=1}^{n}\log\left(1+e^{-y_i(\boldsymbol{\beta}^\top\mathbf{x}_i+\beta_0)+\rho\|\boldsymbol{\beta}\|_q}\right).$$

(e) (6 points) Now let the uncertainty sit in the labels, where the adversary may flip at most Γ of them: $\mathcal{U}_y = \{\boldsymbol{\Delta}\mathbf{y} \in \{0, 1\}^n : \sum_{i=1}^n \Delta y_i \le \Gamma\}$. Although this uncertainty set is binary, the robust counterpart of logistic regression against $\mathcal{U}_y$ can be written as a single convex maximization problem containing no integer variables.

Question 2: Stable Classification (40 points)

A classifier is fit on a labelled dataset of n points by training on k of them and validating on the remaining $n-k$. Which k is a matter of chance, and two draws from the same data can return classifiers that validate equally well and yet disagree about which points they put on either side of the hyperplane.

Lecture 4 takes the chance out of the choice. Write $(\mathbf{x}_i, y_i)$, $i \in [n]$, for the labelled points, with $\mathbf{x}_i \in \mathbb{R}^p$ and $y_i \in \{-1, +1\}$, and write the classifier as $(\boldsymbol{\beta}, \beta_0) \in \mathbb{R}^p \times \mathbb{R}$, scoring point i by $\boldsymbol{\beta}^\top \mathbf{x}_i + \beta_0$. Point i then contributes the loss $\ell(y_i,\, \boldsymbol{\beta}^\top \mathbf{x}_i + \beta_0)$, for a loss function ℓ we leave unspecified until part (c). Give each point a binary z_i recording whether it is in the training set, and, for an integer $k \in [n]$, solve

$$\min_{\boldsymbol{\beta},\, \beta_0} \; \max_{\boldsymbol{z} \in \mathcal{Z}} \; \sum_{i=1}^{n} z_i \, \ell(y_i, \boldsymbol{\beta}^\top \mathbf{x}_i + \beta_0), \qquad \mathcal{Z} = \Big\{ \boldsymbol{z} \in \{0,1\}^n : \sum_{i=1}^{n} z_i = k \Big\}, \tag{2.1}$$

so that the classifier is trained against the hardest training set of size k rather than against one drawn at random.

(a) (10 points) Fix $(\boldsymbol{\beta}, \beta_0)$ and abbreviate $c_i = \ell(y_i, \boldsymbol{\beta}^\top \mathbf{x}_i + \beta_0)$. Let

$$\mathcal{P} = \Big\{ \boldsymbol{z} \in \mathbb{R}^n : \sum_{i=1}^{n} z_i = k, \;\; 0 \le z_i \le 1 \;\; \forall i \in [n] \Big\}$$

be the continuous relaxation of $\mathcal{Z}$, and let $c_{(1)} \ge c_{(2)} \ge \cdots \ge c_{(n)}$ denote the losses sorted in decreasing order. Show that

$$\max_{\boldsymbol{z} \in \mathcal{Z}} \sum_{i=1}^{n} z_i c_i \;=\; \max_{\boldsymbol{z} \in \mathcal{P}} \sum_{i=1}^{n} z_i c_i \;=\; \sum_{j=1}^{k} c_{(j)}.$$

(b) (10 points) By part (a) the inner maximization in (2.1) may be taken over $\mathcal{P}$ in place of $\mathcal{Z}$. Take its linear optimization dual and show that (2.1) is equivalent to

$$\begin{aligned} \min_{\boldsymbol{\beta},\, \beta_0,\, \theta,\, \mathbf{u}} \quad & k\theta + \sum_{i=1}^{n} u_i & \\ \text{s.t.} \quad & \theta + u_i \ge \ell(y_i, \boldsymbol{\beta}^\top \mathbf{x}_i + \beta_0), & i \in [n], \\ & u_i \ge 0, & i \in [n]. \end{aligned} \tag{2.2}$$

Now take ℓ to be the misclassification loss

$$\ell(y_i, \boldsymbol{\beta}^\top \mathbf{x}_i + \beta_0) = \mathbf{1}\{ y_i(\boldsymbol{\beta}^\top \mathbf{x}_i + \beta_0) \le 0 \},$$

which charges one unit for a point the classifier puts on the wrong side of the hyperplane and nothing for one it gets right, so that (2.1) minimizes the number of mistakes made on the worst training set of size k. Is (2.2) still a valid reformulation of (2.1)? Is it one you can hand directly to a solver (no need to write down a reformulation)?

(c) (10 points) Now fix the loss to be the hinge loss, so that

$$\ell(y_i, \boldsymbol{\beta}^\top \mathbf{x}_i + \beta_0) = \max\{ 1 - y_i(\boldsymbol{\beta}^\top \mathbf{x}_i + \beta_0),\; 0 \}.$$

Show that (2.2) becomes a linear optimization problem, write it out in full, and say how many continuous variables and constraints it has in terms of n and p.

Suppose instead we had used the logistic loss of Lecture 6, $\ell(y_i, \boldsymbol{\beta}^\top \mathbf{x}_i + \beta_0) = \log\left(1 + e^{-y_i(\boldsymbol{\beta}^\top \mathbf{x}_i + \beta_0)}\right)$. Can (2.2) then be formulated as a linear mixed-integer optimization problem, with binary variables and linear constraints only? Is the problem tractable?

Hint. Lecture 6 runs the same relaxation and dualization against a cardinality-constrained set of label flips, with this loss. What class of problem does it end up with, and what does it report about solving it?

(d) (10 points) Rather than solving (2.2), we may attack (2.1) by cutting planes, as in Lecture 3. Writing the inner maximization in epigraph form gives

$$\min_{\boldsymbol{\beta},\, \beta_0,\, t} \; t \qquad \text{s.t.} \qquad t \;\geq\; \sum_{i=1}^{n} z_i \, \ell(y_i, \boldsymbol{\beta}^\top \mathbf{x}_i + \beta_0) \quad \text{for every } \boldsymbol{z} \in \mathcal{Z}, \tag{2.3}$$

one constraint for each of the $\binom{n}{k}$ points of $\mathcal{Z}$. We solve (2.3) over a subset $\mathcal{Z}_1 \subseteq \mathcal{Z}$, look for a constraint the resulting solution violates, add it, and repeat.

(i) State the separation problem for an incumbent solution $(\boldsymbol{\beta}^1, \beta_0^1, t^1)$ of the restricted problem, and show that part (a) solves it in closed form. Give the complexity of one separation step in terms of n, in big-O notation.

(ii) Give one reason you might prefer this route to solving (2.2) directly.

Question 3: How Few Constraints Does Convexity Need? (30 points)

A convex function is often the right model even when nobody knows its formula: cost curves, production functions and utility surfaces are convex on theory grounds, and we would like to fit that shape directly rather than guess a parametric form. Recall the convex regression formulation from lecture. Given data (x_i, y_i) for $i \in [n]$ with $x_i \in \mathbb{R}^p$, we fit heights $\theta_i \approx f(x_i)$ and a subgradient ξ_i at each x_i by solving

$$\min_{\theta \in \mathbb{R}^n,\; \{\xi_i \in \mathbb{R}^p\}} \quad \frac{1}{2}\|y - \theta\|_2^2 + \frac{1}{2\gamma}\sum_{i=1}^{n} \|\xi_i\|_2^2 \qquad \text{s.t.} \qquad \theta_i + \xi_i^\top (x_j - x_i) \leq \theta_j \quad \forall\, i \neq j. \tag{3.1}$$

The constraints say that the tangent at every point passes below every other point, which is exactly convexity of the fitted function $f(x) := \max_{i \in [n]}\{\theta_i + \xi_i^\top (x - x_i)\}$. There is one of them per *ordered* pair, so (3.1) carries $n(n-1)$ constraints, and with $n = 40$ that is 1560.

This question is about how few of them you actually need.

Access the workbench in the link https://claude.ai/artifact/5xgYMGVNMgdzucSbNRLJ6p. Log in to your MIT Claude Enterprise account and input your MIT ID to access your own instance ($n = 40$ points, $\gamma = 10$), and the workbench solves it on demand. Nothing is submitted there and there is nothing to fill in: it exists so that you can impose a set of constraints and look at what comes out. Every student's data is different, so the numbers you report are your own.

Two figures are always on screen. The *fit* draws the data, the heights the model chose and each point's tangent, ringing every point that some tangent passes above. Below it, *what makes it convex* plots the slopes in order from left to right: a dot for each tangent slope ξ, a square for each chord slope c between consecutive points. The status line reports the objective and how many of the 1560 constraints the solution still breaks.

Throughout, sort the points so that $x_1 < x_2 < \cdots < x_n$, call $(k, k+1)$ an *adjacent* pair, and let

$$h_k = x_{k+1} - x_k > 0, \qquad c_k = \frac{\theta_{k+1} - \theta_k}{h_k},$$

be the gap between consecutive points and the chord slope across it.

Note what is *not* being asked below. The algorithm in lecture imposes one direction on each pair along a chain and then iterates — solve, find what the solution breaks, add those constraints, solve again — so its starting set only has to get the loop going, and never has to be sufficient on its own. Parts (a) and (b) ask the other question: whether some starting set needs *no second solve at all*, and for which problems.

Part (a): one feature (18 points)

Here $p = 1$. Convexity on the adjacent pair $(k, k+1)$ can be written down in more than one way, and the workbench offers four candidates with a button for each:

forward	$\theta_k + \xi_k(x_{k+1} - x_k) \le \theta_{k+1}$
backward	$\theta_{k+1} + \xi_{k+1}(x_k - x_{k+1}) \le \theta_k$
both	both of the above
stepped backwards	$\theta_k + \xi_k(x_k - x_{k+1}) \le \theta_{k+1}$

(i) (4 points) Impose each candidate on every adjacent pair. Which one leaves nothing broken? Report the objective it reaches and the number of constraints it uses, and briefly compare that objective with what all 1560 constraints give.

(ii) (3 points) Underneath the four candidates is a *control*: pairs chosen at random, with no regard to which points they join, in sizes from 39 up to 624. Try a few. Describe what the random sets do to the fit, and explain why buying more of them is not the same as buying the right ones. No need to tabulate the runs — the argument is what is being asked for.

(iii) (11 points) Prove that your answer to (i) is sufficient: with what you chose imposed on every adjacent pair and nothing else, *every* one of the $n(n-1)$ constraints holds at the optimum. Structure the argument in three steps.

(1) Divide what you imposed on a single pair by h_k. What does it say about ξ_k, ξ_{k+1} and c_k, and is any one of those inequalities enough on its own?

(2) That holds for every $k = 1, \dots, n-1$. Lay the $n-1$ results end to end in the order of the points, and say what follows about the two sequences (ξ_k) and (c_k).

(3) Let $i < j$ be two points that are *not* adjacent, so their constraint was never imposed. Show that it holds anyway. You may use that both the height difference and the distance telescope over the links between the two points,

$$\theta_j - \theta_i = \sum_{k=i}^{j-1} c_k h_k, \qquad x_j - x_i = \sum_{k=i}^{j-1} h_k.$$

State where the argument uses that the points are ordered, and say what changes when $i > j$.

Part (b): two features (7 points)

Now $p = 2$. The data are new but the size is not: still $n = 40$ points and $\gamma = 10$, so still 1560 constraints, and the only thing that has changed is the dimension. Each point now carries a supporting *plane*, and there is no left-to-right order. The natural replacement for "adjacent" is the **Delaunay neighbours**: the pairs of points joined by an edge of the Delaunay triangulation of $x_1, \dots, x_n$. The workbench imposes those pairs for you, in *both* directions — the honest analogue of your answer to (a)(i) — so both parts below refer to one fixed constraint set and there is nothing to choose.

(i) (2 points) The workbench draws the worst violation and labels both ends of it. Name the point whose plane overshoots and the point it passes above, and give the size of the gap.

(ii) (5 points) Which step of your proof in (a)(iii) fails here? State exactly what that step needed, and why $\mathbb{R}^2$ does not provide it.

Part (c): generating the constraints (5 points)

No shortcut closes the two-feature problem, so use the general method: solve the reduced problem, find which constraints the solution breaks, add them, and solve again.

Separation answers a question about an *ordered* pair: for each point i it reports the j whose height i's own supporting plane overshoots worst, so what it returns is the single constraint (i, j). That is what `Separate and solve` adds on each press, one per violating point, and it never adds the reverse — the reverse is not the constraint that was reported violated.

What is open is where to *start*. The workbench offers two initializations: the Delaunay neighbours taken both ways, which is the set Part (b) examined, or one way per edge, which is half as many constraints and closer to how the lecture seeds the loop. Pressing either resets the run. Take both to convergence.

(i) (3 points) For *each* of the two initializations, report how many rounds it took, how many constraints you finished with, and what objective it reached. What does the comparison cost and buy?

(ii) (2 points) Your instance has 1560 constraints and solves outright in a couple of seconds, so none of this was necessary for it. How many constraints would (3.1) have at $n = 10{,}000$? Use that number to say what the method is for.